

dj

A solid red arrow pointing to the right, positioned to the left of the text "L'Admin Django".

L'Admin Django



L'Admin DJANGO

- **Présentation**

- Tout projet Django fournit une interface d'administration automatique
- Elle lit les métadonnées de vos modèles pour fournir une interface rapide et centrée sur les modèles où les utilisateurs de confiance peuvent gérer le contenu du site
- Il n'est pas conçu pour servir de base à la construction d'une interface publique complète.

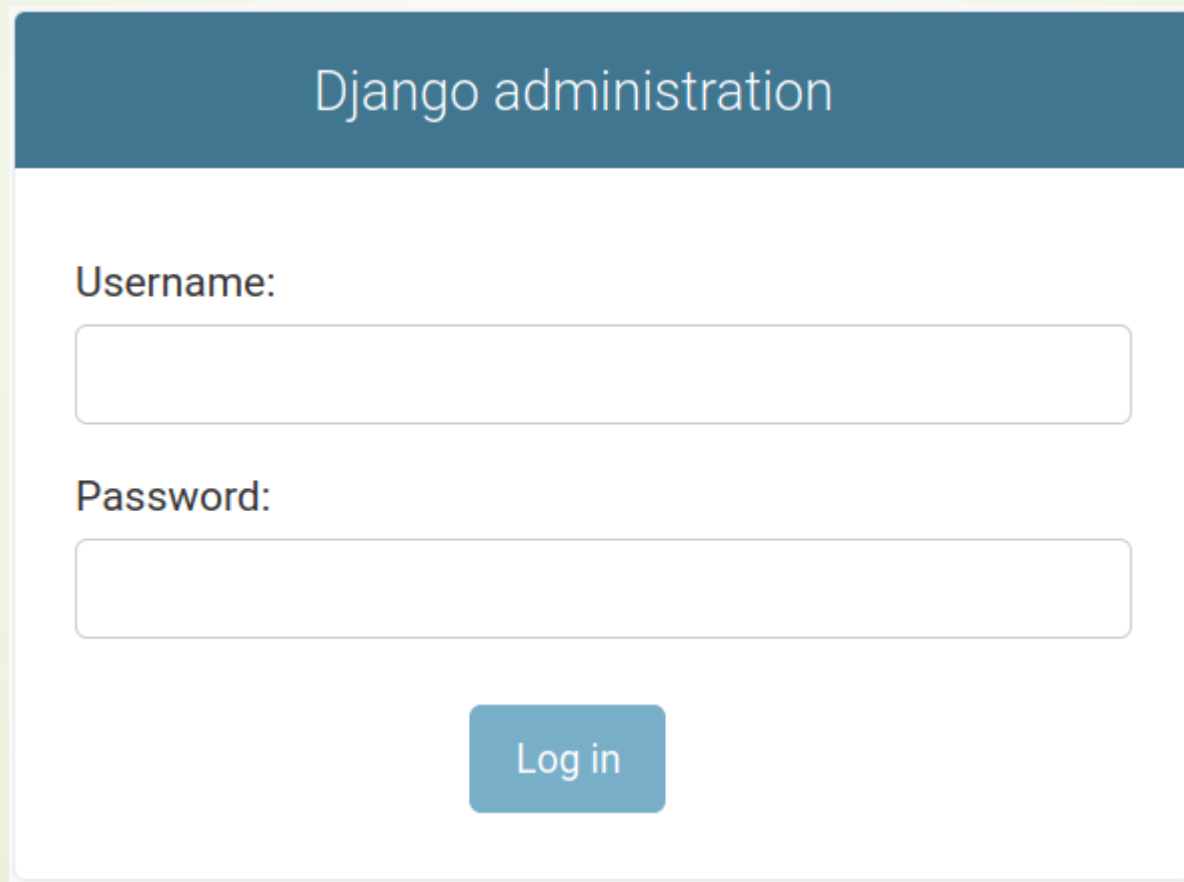
L'Admin DJANGO

- Présentation

- Tout projet Django fournit une interface d'administration automatique
- Elle lit les métadonnées de vos modèles pour fournir une interface rapide et centrée sur les modèles où les utilisateurs de confiance peuvent gérer le contenu du site
- L'application `django.contrib.admin` est installée automatiquement à la création du projet, avec ses dépendances :
 - `django.contrib.auth`
 - `django.contrib.contenttypes`,
 - `django.contrib.sessions`,
 - `django.contrib.messages`
- l'url '`admin/`' est configurée automatiquement
- Il n'est pas conçu pour servir de base à la construction d'une interface publique complète.

L'Admin DJANGO

- Présentation : url /admin/

A screenshot of the Django administration login interface. It features a dark blue header bar with the text "Django administration" in white. Below the header, on a white background, are the labels "Username:" and "Password:" in a dark grey font. Each label is followed by a white rectangular input field with a thin grey border. At the bottom center of the form is a blue rectangular button with the text "Log in" in white.

Django administration

Username:

Password:

Log in

L'Admin DJANGO

- Présentation : créer un superutilisateur

```
py manage.py createsuperuser
```

Username: admin

Email address: admin@example.com

Password: *****

Password (again): *****

Superuser created successfully.

L'Admin DJANGO

- Présentation : créer un superutilisateur

Django administration

WELCOME, ADMIN.

Site administration

AUTHENTICATION AND AUTHORIZATION

Groups

[+ Add](#) [Change](#)

Users

[+ Add](#) [Change](#)

Recent actions

My actions

None available

L'Admin DJANGO

- Ajouter un modèle d'une app à l'admin
- Django utilise la fonction `autodiscover()` pour rechercher des modèles à ajouter dans les fichiers `admin.py` des applications, au moyen de la fonction `admin.site.register`

dans app/admin.py

```
from django.contrib import admin
```

```
from .models import Modele
```

```
# ajout classique  
admin.site.register(Modele)
```

L'Admin DJANGO

- Ajouter un modèle d'une app à l'admin

Site administration

AUTHENTICATION AND AUTHORIZATION

Groups

[+ Add](#) [✎ Change](#)

Users

[+ Add](#) [✎ Change](#)

WEBSITE

Articles

[+ Add](#) [✎ Change](#)

L'Admin DJANGO

- Ajouter un enregistrement du modèle

Add article

Texte:

Date de publication:

Date: Today 

Time: Now 

Note: You are 1 hour ahead of server time.

L'Admin DJANGO

- Ajouter un enregistrement du modèle

✓ The article "Article object (1)" was added successfully.

Select article to change

Action: 0 of 1 selected

☐	ARTICLE
☐	Article object (1)

1 article

L'Admin DJANGO

- Modification d'un enregistrement

Change article HISTORY

Texte:

ekfnzekfn
renergnerkng
elrngekrjgb

L'Admin DJANGO

- Historique des modifications d'un enregistrement

Change history: Article object (1)

DATE/TIME	USER	ACTION
Oct. 29, 2020, 6:36 p.m.	admin	Added.

L'Admin DJANGO

- **Personnalisation : les classes ModelAdmin**
- Les classes ModelAdmin permettent de surcharger les modèles à des fins de personnalisations de leur gestion dans l'admin
- On peut utiliser register sous forme de décorateur de ces classes pour les ajouter dans l'admin

dans app/admin.py

```
from django.contrib import admin  
from .models import Article
```

```
@admin.register(Article)  
class ArticleAdmin(admin.ModelAdmin):  
    pass
```

L'Admin DJANGO

- **ModelAdmin : date_hierarchy**
- le nom d'un champ **DateField** ou **DateTimeField** de votre modèle, et la page de liste pour modification inclura une navigation par filtres sélectifs basés sur les dates en fonction de ce champ.

```
class ArticleAdmin(admin.ModelAdmin):  
    date_hierarchy = 'date_de_publication'
```

Select article to change

◀ 2020 **October 20** October 29

Action: 0 of 2 selected

☐	ARTICLE
☐	Article object (2)
☐	Article object (1)

L'Admin DJANGO

- `ModelAdmin : empty_value_display`
- La valeur affichée des champs vides.

```
class ArticleAdmin(admin.ModelAdmin):  
    empty_value_display = '-empty-'
```

L'Admin DJANGO

- **ModelAdmin** : gestion des champs affichés

```
class ArticleAdmin(admin.ModelAdmin):  
    # ordre d'affichage des champs dans l'admin  
    fields = ('titre', 'texte', 'date_de_publication')  
  
    # exclure des champs  
    exclude = ('titre',)  
...  
# disposition des champs dans l'interface  
class ArticleAdmin(admin.ModelAdmin):  
    fieldsets = (  
        (None, {  
            'fields': ('titre',)  
        }),  
        ('contenu', {  
            'classes': ('collapse',),  
            'fields': ('texte', 'date_de_publication'),  
        }),  
    )
```

L'Admin DJANGO

- ModelAdmin : gestion des champs affichés

Change article

Titre:

contenu [\(Show\)](#)

[Delete](#) [Save and add another](#)

L'Admin DJANGO

- ModelAdmin : gestion des listes de champs

champs affichés sur la liste
`list_display = ('titre', 'date_de_publication')`

◀ 2020 October 20 October 29

Action: 0 of 2 selected

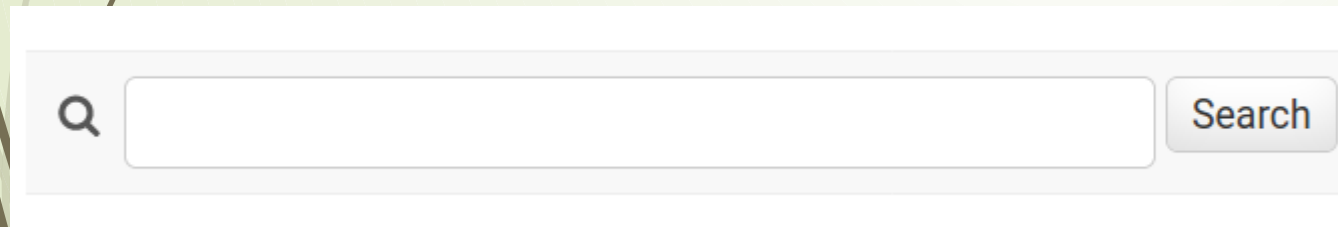
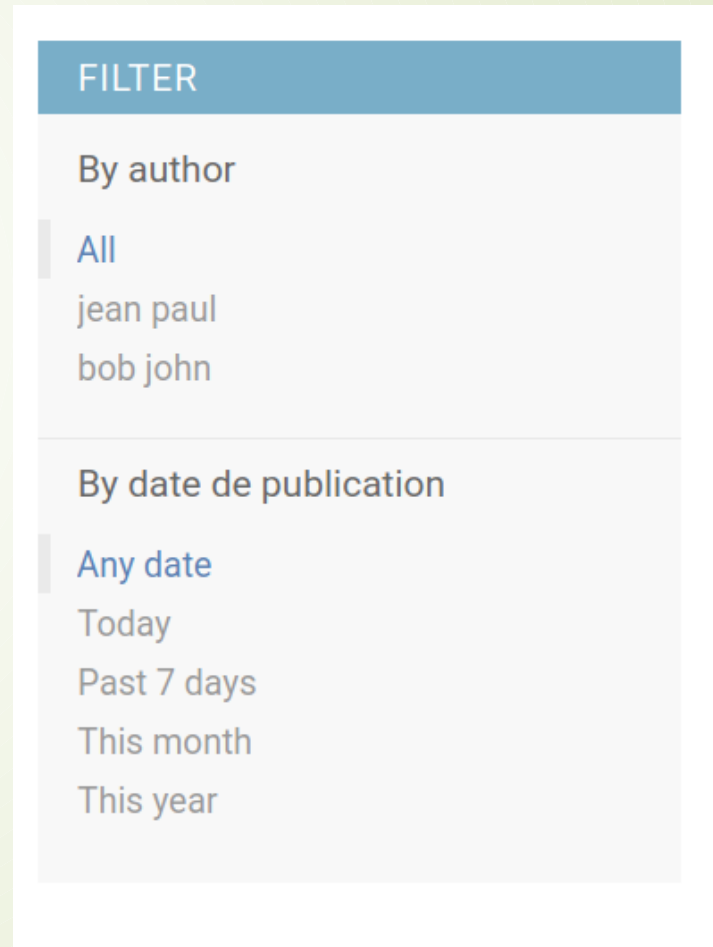
☐	TITRE	DATE DE PUBLICATION
☐	-empty-	Oct. 20, 2020, 7:11 p.m.
☐	-empty-	Oct. 29, 2020, 6:36 p.m.

L'Admin DJANGO

- ModelAdmin : gestion des listes de champs

```
# filtres sur la liste  
list_filter = ('author', 'date_de_publication')
```

```
# barre de recherche  
search_fields = ['texte', 'titre']
```

A screenshot of a search bar from the Django Admin interface. It features a magnifying glass icon on the left, a text input field in the center, and a 'Search' button on the right.A screenshot of the 'FILTER' sidebar from the Django Admin interface. It has a blue header with the word 'FILTER'. Below the header, there are two sections. The first section is titled 'By author' and contains a list of filters: 'All' (highlighted with a blue bar), 'jean paul', and 'bob john'. The second section is titled 'By date de publication' and contains a list of filters: 'Any date' (highlighted with a blue bar), 'Today', 'Past 7 days', 'This month', and 'This year'.

L'Admin DJANGO

- ModelAdmin : personnalisation des champs

```
formfield_overrides = {  
    models.TextField: {'widget': RichTextWidget},  
}
```

Texte:

B	<i>I</i>			<u>I</u> _x
----------	----------	---	---	-----------------------

cpep,evprnvernkv ekvoekrnvoerknv eokrnvoernonv|

P

<https://pypi.org/project/django-richtextfield/>

L'Admin DJANGO

- ModelAdmin : Gestion de la Foreign Key

- L'affichage d'un objet en tant que chaîne de caractère se fait via la méthode `__str__()`

Titre:	blabla
Author:	Author object (1)  

Titre:	
Author:	jean paul  

L'Admin DJANGO

- **ModelAdmin : Créer des actions**

une action est une fonction avec les paramètres modeladmin, request et queryset

```
def make_published(modeladmin, request, queryset):  
    queryset.update(status='p')
```

```
make_published.short_description = "publier les articles"
```

placer la fonction comme action

```
class ArticleAdmin(admin.ModelAdmin):  
    actions = [make_published]
```

Action: 0 of 1 selected

☐	TITRE	DATE DE PUBLICATION	AUTHOR	STATUS
☐	blo blo blo	Oct. 27, 2020, 9:27 p.m.	bob john	Published